

Lista Avaliativa 6 - Respostas

1)

Para gerenciar nomes de arquivos longos ou de tamanho variável, os sistemas de arquivos utilizam principalmente duas abordagens. A primeira consiste em fazer com que cada entrada no diretório tenha um tamanho variável. Nesse caso, a entrada possui um cabeçalho de tamanho fixo com os atributos do arquivo, seguido pelo nome do arquivo com seu tamanho real. O problema dessa técnica é que, ao remover um arquivo, ela cria um espaço vazio de tamanho variável no diretório, o que pode levar à fragmentação interna do próprio arquivo de diretório, dificultando o reuso desse espaço.

Uma solução mais eficiente, e comumente usada, é manter as entradas de diretório com um tamanho fixo, mas armazenar os nomes dos arquivos em uma área separada no final do arquivo de diretório, conhecida como heap. Nessa abordagem, cada entrada de diretório contém um ponteiro para o local no heap onde o nome do arquivo está de fato armazenado. Isso resolve o problema da fragmentação, pois todas as entradas de diretório têm o mesmo tamanho, facilitando a remoção e a adição de novos arquivos.

2)

O journaling é uma técnica usada para garantir a consistência do sistema de arquivos em caso de falhas, como uma queda de energia. Operações complexas, como a exclusão de um arquivo, envolvem múltiplos passos (remover a entrada do diretório, liberar o i-node, retornar os blocos de dados para a lista de blocos livres). Se o sistema falhar no meio dessa sequência, o sistema de arquivos pode ficar em um estado inconsistente.

Para evitar isso, um sistema com journaling primeiro escreve em uma área especial do disco, chamada journal ou diário, um registro de todas as operações que ele pretende executar. Apenas depois que esse registro é salvo com sucesso no disco, o sistema de arquivos começa a realizar as modificações nos dados e metadados. Após a conclusão de todos os passos, a entrada no journal é marcada como completa. Se o sistema falhar e reiniciar, ele primeiro verifica o journal. Se encontrar uma operação que não foi concluída, ele a executa novamente a partir das informações do registro para garantir que o sistema de arquivos retorne a um estado consistente e íntegro.

3)

A alocação baseada em i-nodes é um método para organizar os blocos de um arquivo. Nesse esquema, a entrada de um diretório para um arquivo contém apenas o nome do arquivo e o número do i-node correspondente. O i-node é uma estrutura de dados que armazena todos os metadados do arquivo, como permissões, datas e, mais importante, os endereços dos blocos de disco onde os dados do arquivo estão armazenados.

Para arquivos pequenos, o i-node contém uma lista de ponteiros diretos que apontam para os blocos de dados. Conforme o arquivo cresce, essa estrutura se expande para suportar arquivos maiores. Para isso, o i-node utiliza ponteiros indiretos. Um ponteiro de indireção simples aponta para um bloco de disco que, em vez de conter dados, contém uma lista de ponteiros para os blocos de dados. Para arquivos ainda maiores, podem ser usados ponteiros de indireção dupla (que aponta para um bloco de ponteiros que apontam para outros blocos de ponteiros) e até tripla. Essa estrutura em níveis permite que o sistema de arquivos acesse eficientemente tanto arquivos pequenos quanto muito grandes.

4)

A tabela FAT é uma estrutura de dados que funciona como um mapa para o disco, organizando os arquivos como uma lista encadeada de blocos. Em vez de armazenar o ponteiro para o próximo bloco dentro do bloco de dados anterior, todos os ponteiros são centralizados em uma única tabela, a FAT, que é mantida na memória.

A FAT é essencialmente um grande vetor, onde cada índice corresponde a um bloco de dados no disco. A entrada de diretório de um arquivo aponta para o número do seu primeiro bloco. Para encontrar o restante do arquivo, o sistema operacional usa esse número como um índice na FAT. O valor contido nessa posição da tabela é o número do próximo bloco do arquivo. Esse processo se repete, seguindo a cadeia de ponteiros dentro da FAT, até que um valor especial de "fim de arquivo" seja encontrado, indicando que não há mais blocos. Esse método permite um acesso aleatório relativamente rápido, pois toda a cadeia de blocos pode ser percorrida na memória, sem a necessidade de ler cada bloco de dados do disco.

5)

As Listas de Controle de Acesso (ACLs) são um mecanismo que oferece um controle de permissões mais detalhado e flexível do que o modelo padrão de permissões do UNIX (proprietário, grupo e outros). Enquanto o modelo tradicional é limitado, as ACLs permitem definir permissões para múltiplos usuários e grupos específicos em um mesmo arquivo ou diretório.

Uma ACL é uma lista de entradas associada a um arquivo. Cada entrada, chamada de Entrada de Controle de Acesso (ACE), especifica um usuário ou grupo e as permissões (leitura, escrita, execução) que são concedidas ou negadas a ele. Quando um processo tenta acessar um arquivo protegido por uma ACL, o sistema operacional primeiro verifica as permissões tradicionais e, em seguida, percorre a ACL para encontrar uma entrada que corresponda ao usuário e aos grupos aos quais ele pertence. Isso possibilita a criação de regras de acesso complexas, que não seriam possíveis de implementar apenas com o sistema de permissões básico.

6)

```
#include <stdio.h>
#include <sys/statvfs.h>
#include <stdlib.h>

// Função para formatar bytes em um formato legível (KB, MB, GB)
void format_bytes(unsigned long long bytes, char *buffer, int size) {
    const char *suffixes[] = {"B", "KB", "MB", "GB", "TB"};
    int i = 0;
    double d_bytes = bytes;

    while (d_bytes >= 1024 && i < 4) {
        d_bytes /= 1024;
        i++;
    }

    snprintf(buffer, size, "%.2f %s", d_bytes, suffixes[i]);
}
```

```

int main(int argc, char *argv[]) {
    // Validação dos Argumentos de Entrada
    if (argc != 2) {
        fprintf(stderr, "Uso: %s <caminho_do_sistema_de_arquivos>\n",
argv[0]);
        exit(1);
    }

    // Declaração da Estrutura e Chamada da Função
    struct statvfs buf;

    if (statvfs(argv[1], &buf) != 0) {
        perror("Erro ao chamar statvfs");
        exit(2);
    }

    // Cálculo e Apresentação das Informações
    unsigned long long total_space = buf.f_blocks * buf.f_frsize;
    unsigned long long free_space = buf.f_bfree * buf.f_frsize;
    unsigned long long available_space = buf.f_bavail * buf.f_frsize;

    char total_str[32], free_str[32], avail_str[32];
    format_bytes(total_space, total_str, sizeof(total_str));
    format_bytes(free_space, free_str, sizeof(free_str));
    format_bytes(available_space, avail_str, sizeof(avail_str));

    printf("Informações do sistema de arquivos para: %s\n", argv[1]);
    printf("-----\n");
    printf("Tamanho do bloco fundamental (f_frsize):      %lu bytes\n",
buf.f_frsize);
    printf("ID do sistema de arquivos (f_fsid):              %lu\n",
buf.f_fsid);
    printf("Número máximo de caracteres em nome de arq.: %lu\n",
buf.f_namemax);
    printf("\n");
    printf("Espaço Total:                %s (%llu bytes)\n", total_str,
total_space);
    printf("Espaço Livre:                %s (%llu bytes)\n", free_str,
free_space);
    printf("Espaço Disponível (não-root): %s (%llu bytes)\n", avail_str,
available_space);
    printf("\n");
    printf("Total de nós-i (inodes):      %llu\n", (unsigned long
long)buf.f_files);
    printf("Nós-i (inodes) livres:       %llu\n", (unsigned long
long)buf.f_ffree);

    return 0;
}

```

7)

Programa 1:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    const char *nome_arquivo = "numeros.bin";

    FILE *arquivo = fopen(nome_arquivo, "wb");

    if (arquivo == NULL)
    {
        perror("Erro ao abrir o arquivo para escrita");
        return 1;
    }

    printf("Arquivo '%s' aberto para escrita.\n", nome_arquivo);

    for (int i = 1; i <= 30; i++)
    {
        if (fwrite(&i, sizeof(int), 1, arquivo) != 1)
        {
            fprintf(stderr, "Erro ao escrever o número %d no arquivo.\n",
i);
            fclose(arquivo);
            return 1;
        }P
    }

    printf("30 números inteiros foram escritos com sucesso no arquivo
'%s'.\n", nome_arquivo);

    fclose(arquivo);

    return 0;
}
```

Programa 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/uio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
```

```
const char *nome_arquivo = "numeros.bin";

int fd = open(nome_arquivo, O_RDONLY);

if (fd == -1) {
    perror("Erro ao abrir o arquivo para leitura");
    return 1;
}

off_t offset = 9 * sizeof(int);
if (lseek(fd, offset, SEEK_SET) == -1) {
    perror("Erro ao posicionar o cursor no arquivo");
    close(fd);
    return 1;
}

int numeros_lidos[8];

struct iovec iov[8];

for (int i = 0; i < 8; i++) {
    iov[i].iov_base = &numeros_lidos[i];
    iov[i].iov_len = sizeof(int);
}

ssize_t bytes_lidos = readv(fd, iov, 8);

if (bytes_lidos == -1) {
    perror("Erro ao ler o arquivo com readv");
    close(fd);
    return 1;
}

printf("Leitura realizada com sucesso a partir do 10º inteiro.\n");
printf("Total de bytes lidos: %ld\n", bytes_lidos);
printf("Números lidos do arquivo:\n");

for (int i = 0; i < 8; i++) {
    printf("Buffer %d: %d\n", i + 1, numeros_lidos[i]);
}

close(fd);

return 0;
}
```